



**SZÉCHENYI
EGYETEM**
UNIVERSITY OF GYŐR —

[GKNB_INTM005] – Projektmunka 2

Széchenyi István Egyetem – Tantárgyi
Tudásbázis

Varga Zoltán, Homoki Tamás, Győri Máté,
Németh János, Farkas Bence, Hegedűs Dávid

Tartalom

A projekt ötletének eredete	3
A projekt sokoldalúsága	3
Felhasználói szintek.....	4
Database.....	5
1. Áttekintés.....	5
2. Táblák részletes ismertetése	5
3. Táblák közötti függőségek diagramja	7
4. Elsődleges kulcsok.....	8
5. Relációs séma	8
6. Jövőbeli tervek.....	8
Front-End.....	9
1. Hogyan kezdődött?	9
2. Hogyan kezdtünk neki?.....	9
Back-End	17
Fejlesztési eszközök	28

A projekt ötletének eredete

Mindannyian sokat gondolunk egy olyan oldalra, amely egységesíti és a jegyzeteket, oktatási anyagokat az adott tantárgyakhoz véleményezési lehetőséget nyújt különböző módokon. Elsősorban az fórummal azt szeretnénk elérni, hogy az oktatók, tanársegédek, auditorok, egyéb hallgatóknak sokat segítő külsős tanárok és maguk a hallgatók is megtudják osztani azokat a jegyzetek, amelyek segítik a hallgatót abban, hogy a lehető legjobb eredményt tudják nyújtani a gyakorlatokon, zárthelyiken, vizsgákon és ami a legfontosabb tanulmányaik után az éles helyzetekben is. Az oldal ötlete előtt is megfogalmazódott bennünk az, hogy minden hallgató és minden oktató máshogy tudja átadni és feldolgozni a kapott információkat a jegyzetektől, videókból, hanganyagokból és kötelező olvasmányokból. Egyszóval úgy ítéltük meg, hogy minden lehetőséget biztosítani kell a hallgatóknak arra, hogy sikeresen, ne csak megtanulják, hanem meg is értsék az órák alatt elhangzott anyagot vagy akár jegyzetektől, videókból megismert információkat, elsajátítsák és a munkaerőpiacon már hasznosítani tudják azt.

A projekt sokoldalúsága

A projekt során felmértünk bizonyos oktatók, auditorokat, tanársegédek, régi és új hallgatókat, hogy számukra mit jelentene egy ilyen fórum. A visszajelzések először vegyesek voltak, majd miután pontosítottuk a projekt és az oldal célját szimpatikusabbá vált mindenkinek.

Több aggály felmerült a projekt tervezése során többek között:

- Miért kell még egy weboldal a többi mellé?
- Nem lenne elég PlugIN-okat fejleszteni a meglévő oldalakhoz?
- Milyen bejelentkezési lehetőséget nyújtanánk a felhasználóknak?
- Hogyan lehetne kiszűrni a trollkodást / Hogyan lehetne moderálni az oldalt?

Ezek voltak a főbb és többször visszatérő kérdések a dokumentáció alatt ezekre mind kitérünk és alátámasztjuk ezeket, hogy hogyan is tud egy ilyen oldal ilyen releváns lenni mindenki számára. Hogyan lesz ez számot tevő tényező az RS1.sze.hu-s oldalak, Moodle, Videótórium, külsős oktatók által biztosított oldalak mellett, valamint egyéb olyan oldalakat tekintve, amelyek akár az egyetem üzemeltetése alatt álnak vagy akár más portálokon érhetőek el.

Összességében azt mondhattuk el a felmérések és egyéni beszélgetések végén, hogy az általunk felmért embereket sikerült meggyőznünk és bebizonyítanunk nekik, hogy egy ilyen oldalnak lehet keresnivalója az egyetem életében.

Felhasználói szintek

Az oldalon két különböző felhasználói szintet kezelünk.

- User
- Admin

Az admin felhasználónk egy igazi Jolly Joker kártya hiszen ő felelős az oldal hiba javításáért és zavartalan működésének biztosításáért, felhasználókat, témákat törölhet, az oldal kinézetét módosíthatja, eléréseket beállíthatja. Az oldalon történő hibák kezelésénél ő avatkozik be a hibák javításába. Ő felelős a szetf.help@gmail.com -ra beérkező panaszok, javaslatok kezeléséért és megvalósításáért, esetleges felhasználó törlésekért, téma módosításokért.

Database

1. Áttekintés

Ez az adatbázis egy egyetemi rendszert modellez, amely támogatja az intézmény adminisztratív és oktatási folyamatait. Tartalmaz adatokat az egyetemi karokról, szakokról, tantárgyakról, valamint felhasználókról és azok tevékenységeiről, például témák, értékelések és fájlok kezeléséről.

Célja:

Az oktatási adminisztrációs folyamatok digitalizálása és optimalizálása.
Témák, felhasználók kezelése, keresőmotor működésének megfelelő biztosítása, illetve egyéb felhasználó számára egyszerűnek tűnő feladatok elvégzése. (pl.: Jelszó emlékeztető)

2. Táblák részletes ismertetése

2.1. *Faculties (Karok)*

- Funkció: Az egyetem karjainak listáját tartalmazza.
- Használati forgatókönyvek:
 - Új karok hozzáadása, például új fakultások létrehozásakor.
 - Egy kar összes szakjának lekérdezése.
- Kulcsfontosságú attribútumok:
 - `faculty_name`: Egyedi és kötelezően megadandó. Ez határozza meg a kart.
 - `description`: Opcionális, további információkat nyújt a kar funkciójáról vagy specializációjáról.

2.2. *Majors (Szakok)*

- Funkció: Karokhoz kapcsolt szakok tárolása.
- Fontos jellemzők:
 - `faculty_id`: Kapcsolatot hoz létre a faculties táblával.
 - `major_name`: Egyedi név, amely egy szakot azonosít.

2.3. *Users (Felhasználók)*

- Funkció: Felhasználók (hallgatók, adminok) adatainak kezelése.
- Fontos jellemzők:
 - `email`: Egyedi azonosítónak szolgál a bejelentkezéshez.
 - `role`: Meghatározza a felhasználó szerepét és jogosultságait.

2.4. *Courses (Tantárgyak)*

- Funkció: Tantárgyak nyilvántartása, amelyek egy adott szakhoz kapcsolódnak.
- Használat:
 - Tantárgyak listázása szak szerint.
 - Tantárgyak létrehozása, szerkesztése és törlése.

2.5. *Topics (Témák)*

- Funkció: Tantárgyakhoz tartozó témák kezelése, mint például előadások, jegyzetek vagy projektek.

- Fontos attribútumok:
 - `course_id`: Kapcsolatot hoz létre a `courses` táblával.
 - `user_id`: Annak a felhasználónak az azonosítója, aki létrehozta a témát.

2.6. Ratings (Értékelések)

- Funkció: Felhasználók véleményezhetik a témákat pozitív vagy negatív értékeléssel.
- Fontos attribútumok:
 - `rating`: Az értékelés típusa, csak pozitív vagy negatív értékeket vehet fel.

2.7. Files (Fájlok)

- Funkció: Témákhoz kapcsolódó fájlok, anyagok tárolása, például tananyagok vagy segédletek.
- Fontos attribútumok:
 - `file_type`: A fájl típusát jelöli, például "vizsga", "anyag", "segédlet".

2.8. Enrollments (Beiratkozások)

- Funkció: Felhasználók tantárgyakra való feliratkozásának nyilvántartása.
- Fontos attribútumok:
 - `enrolled_at`: A beiratkozás időpontját rögzíti.

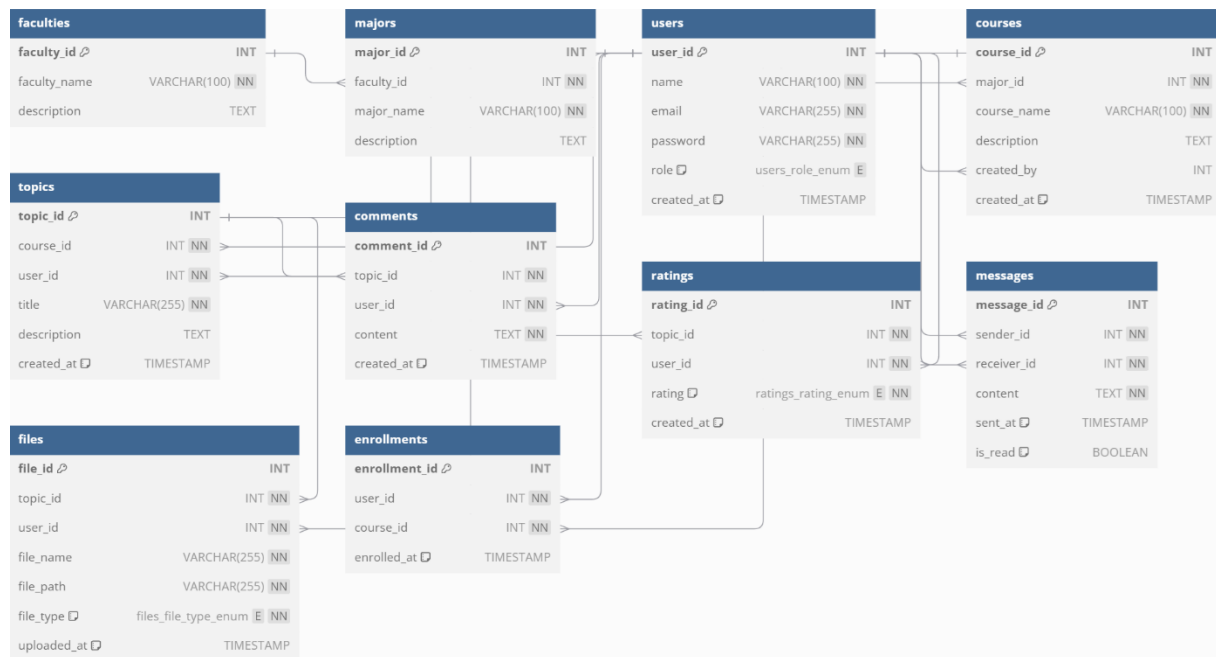
2.9. Comments (Kommentek)

- Funkció: Témákhoz kapcsolódó hozzászólások tárolása.
- Fontos attribútumok:
 - `content`: A komment szöveges tartalma.
 - `created_at`: A komment létrehozásának dátuma.

2.10. Messages (Üzenetek)

- Funkció: Privát üzenetküldés a felhasználók között.
- Fontos attribútumok:
 - `is_read`: Az üzenet olvasottsági állapotát jelzi.

3. Táblák közötti függőségek diagramja



4. Elsődleges kulcsok

- faculty_id
- major_id
- user_id
- topic_id
- comment_id
- course_id
- rating_id
- message_id
- file_id
- enrollment_id

5. Relációs séma

faculties	faculty_id, faculty_name, description
majors	major_id, faculty_id, major_name, description
users	user_id, name, e-mail, password, role, created_at
courses	course_id, major_id, course_name, description, created_by, created_at
topics	topic_id, course_id, user_id, title, description, created_at
comments	comment_id, topic_id, user_id, content, created_at
ratings	rating_id, topic_id, user_id, rating, created_at
messages	message_id, sender_id, receiver_id, content, sent_at, is_read
files	file_id, topic_id, user_id, file_name, file_path, file_type, uploaded_at
enrollments	enrollment_id, user_id, course_id, enrolled_at

6. Jövőbeli tervek

Az adatbázis további fejlesztése érdekében a következő funkciókat és fejlesztéseket terveztük megvalósítani, amelynek célja, hogy az adatbázis hatékonyabbá, biztonságosabbá és felhasználóbarátabbá váljon.

6.1 Kommentek és üzenetek fejlesztése:

- A kommentek és üzenetek kezelésének továbbfejlesztése, például lehetőségek biztosítása a szerkesztésre vagy törlésre.
- Olvasottsági státusz pontosabb követése az üzenetek esetében.

6.2 Jelszó titkosítása:

- A felhasználói jelszavak titkosításának bevezetése biztonságos algoritmusokkal, hogy növeljük az adatbiztonságot.

6.3 Felhasználói aktivitás követése:

- Felhasználók aktivitásának nyomon követése, például, hogy mikor és milyen műveleteket hajtottak végre (bejelentkezések, hozzászólások, fájlfeltöltések).

6.4 Adatok rendezése és szűrése:

- Különböző rendezési és szűrési lehetőségek bevezetése a lekérdezések során (pl. dátum szerint növekvő vagy csökkenő sorrend, relevancia alapú szűrés).

6.5 Naplózás:

- Egy audit-log rendszer kiépítése, amely automatikusan naplózza a kritikus adatbázis-műveleteket (pl. adatbeszúrások, frissítések, törlések), az esetleges hibák könnyebb nyomon követése és visszaállítása érdekében.

Front-End

1. Hogyan kezdődött?

A projekt indulásakor számtalan online és személyes megbeszélés, ötletelés, tervezés ment végbe. Több elképzelés született, hogy hogyan nézhetne ki a felület (lista nézet, csempe nézet, stb.), viszont mindegyik megoldás mögött voltak pozitív és negatív érvek.

- Felhasználói szempontból melyik hívogatóbb?
- Melyiket könnyebb/egyértelműbb kezelni?
- Nem lenne sok egy oldalon megjeleníteni minden témát?

A tervezés során több alapelvet lefektettünk, amelyek úgymond sarokkövei voltak a projektnek.

- Ne legyen vizuálisan romboló, de mégis legyen látványos.
- A felhasználók szempontjából legyen egyértelmű minden megjelenített dolog.
- Használjuk az egyetem weboldalaihoz használt színeket, arculati elemeket.

Ekkor döntöttünk úgy, hogy a csempe nézetet fogjuk alkalmazni, az oldal színei pedig az egyetem által használt/működtetett oldalak és alkalmazások témájához fog illeszkedni. Ehhez a **#283454** színkóddal rendelkező színt használtuk.

2. Hogyan kezdtünk neki?

Tanulmányaink során nem használtunk HTML és CSS programozást, így első lépésként el kellett sajátítanunk ezen nyelveket. Videós anyagok, online kurzusok keretein belül sikerült olyan mennyiségű és minőségű ismeretet magunkra szedni, hogy magabiztosan neki tudtunk kezdeni a projektnek.

A folyamatos megbeszélések alkalmával megterveztük, hogy milyen pontokat valósítsunk meg. Miután a csempe nézet opciót választottuk, a következő lépés az volt, hogy miket jelenítsünk meg. Ekkor jött az a gondolat, hogy legyen egy főoldal, amely az egyetemünk 9 karját jeleníti meg. Fontos volt számunkra, hogy a csempék kellően el legyenek különítve, illetve jól látható legyen, hogy hova lehet kattintani a felhasználónak. Nem túl vastag, de nem is nagyon vékony margókat alkalmaztunk, illetve a megjelenítés szempontjából a szöveget vertikálisan és horizontálisan is centralizáltuk.

```
.grid {  
  display: grid;  
  grid-template-columns: repeat(auto-fill, minmax(220px, 1fr));  
  gap: 20px;  
  padding-top: 5px;  
  padding-bottom: 45px;  
}
```

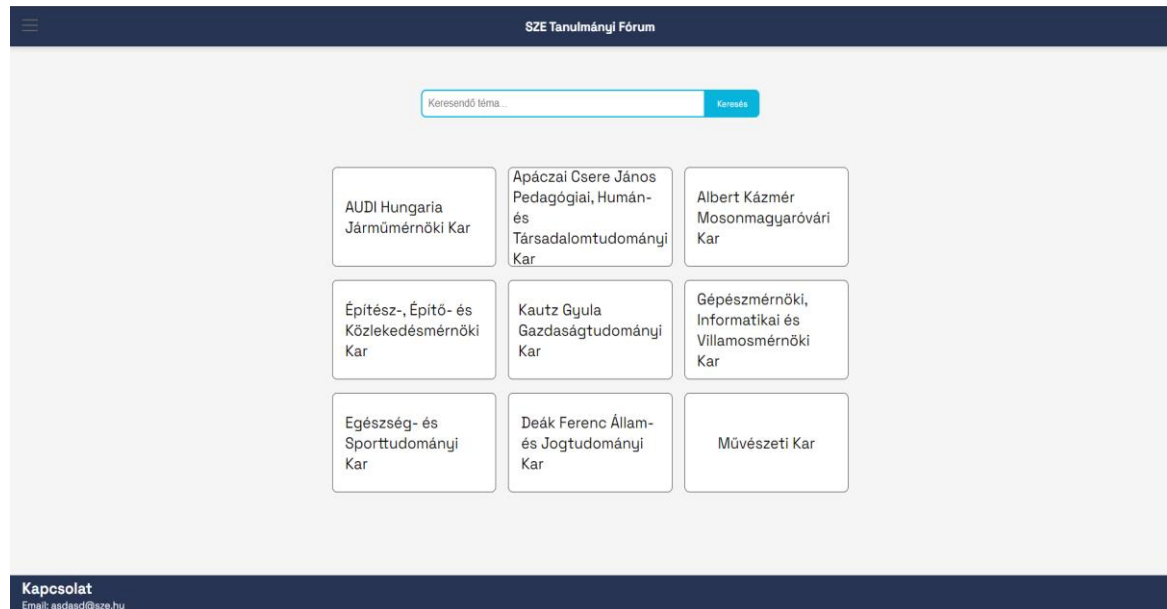
1. ábra Főoldal grid felépítése

Ennek megvalósításakor több ötletünk is támadt, hogy esetleg egyedi háttereket tervezzünk a karoknak, vagy az egyedi színeikkel lássuk el őket. Végül lett egy harmadik megoldás, amelyet a legelegánsabbnak tartottunk, mégpedig azt, hogy amikor a kurzort rávisszük az adott kar csempéjére, akkor halványan, de mégis láthatóan jelenjen meg a csempében a kar színe. Amikor pedig elvisszük róla a fókusz, akkor álljon vissza a standard fehér háttérre.

```
.tile:nth-child(1):hover {  
  background-color: #B7B2AC;  
}  
.tile:nth-child(2):hover {  
  background-color: #2A8C46;  
}  
.tile:nth-child(3):hover {  
  background-color: #D67C1C;  
}  
.tile:nth-child(4):hover {  
  background-color: #AE791D;  
}  
.tile:nth-child(5):hover {  
  background-color: #357ABE;  
}  
.tile:nth-child(6):hover {  
  background-color: #A1C038;  
}  
.tile:nth-child(7):hover {  
  background-color: #AC112B;  
}  
.tile:nth-child(8):hover {  
  background-color: #EEC344;  
}  
.tile:nth-child(9):hover {  
  background-color: #7765A6;  
}
```

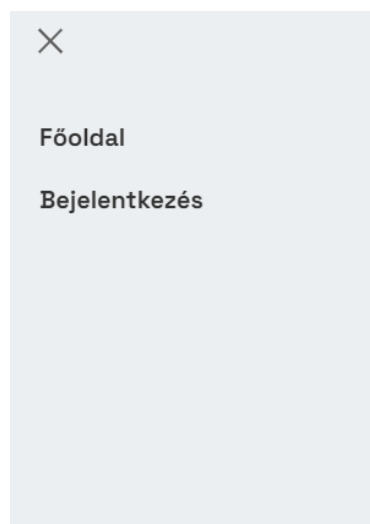
2. ábra Grid színeinek beállítása

Miután elkészültek a csempék, a következő lépésként a header és a footer elkészítését vettük sorba. Egyértelmű volt, hogy méretük egyezzen, mivel így egy kellemes, egységes, igényes látványt nyújt a felhasználónak. A header-ben mindenképpen kötelezőnek véltük a „SZE Tanulmányi Fórum” szöveg feltüntetését, hogy a user-ek tudják, hogy mit is használnak pontosan. A header és a footer minden, az oldal további funkcióinál egységesen így fog megjelenni, mint kinézetében, mint tartalmában.

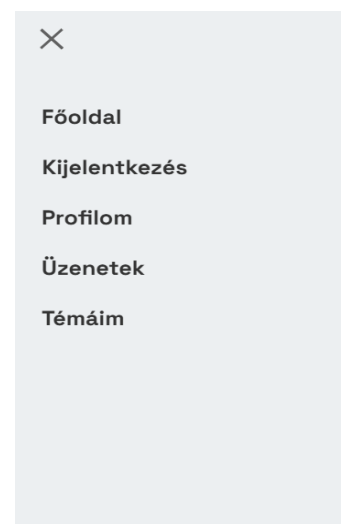


3. ábra Főoldal

Továbbá a header bal oldalán elhelyeztünk egy „hamburger” menüt, amelyben az oldalhoz tartozó különböző funkciók jelennek meg listászerűen felsorolva. Oldalunkon alkalmaztunk egy bejelentkezési rendszert, amellyel kapcsolatban fontos kiemelni, hogy miután a felhasználó bejelentkezett az oldalra, a „hamburger” menüben további funkciók jelennek meg számára.

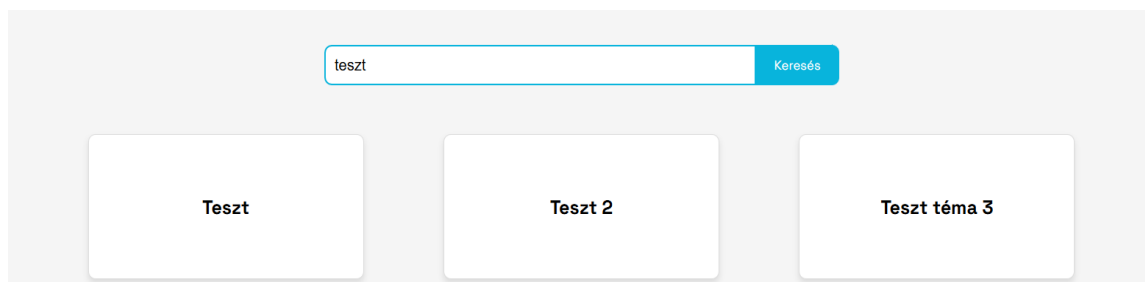


Bejelentkezés előtt



Bejelentkezés után

Nem utolsó sorban az oldalon elhelyezésre került egy kereső sáv, amely az oldalra feltöltött témák között keres. A keresés során a top 3 legpontosabb találatot fogja megjeleníteni számunkra, illetve amennyiben egy adott szöveg részletet tartalmaz a téma neve, akkor is kihozza a megfelelő témákat (pl.: kereső sávba beírjuk, hogy „Tes”, és ki fogja nekünk hozni például a „Teszt1, Tesztelés, Tesztelgetés” téma találatokat). Amennyiben nem talál semmilyen témát a keresés során, azt a hibaüzenetet fogjuk kapni, hogy „Nincs találat a keresett kifejezésre”.

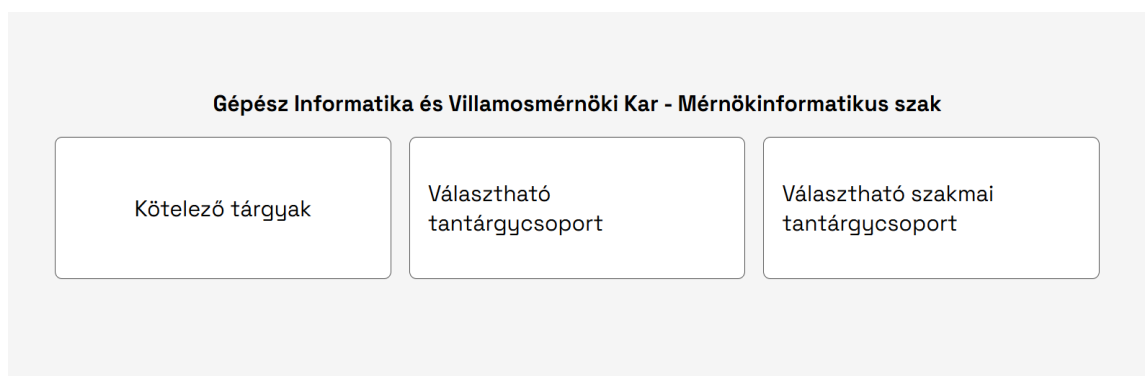


4. ábra Találatok megjelenítve



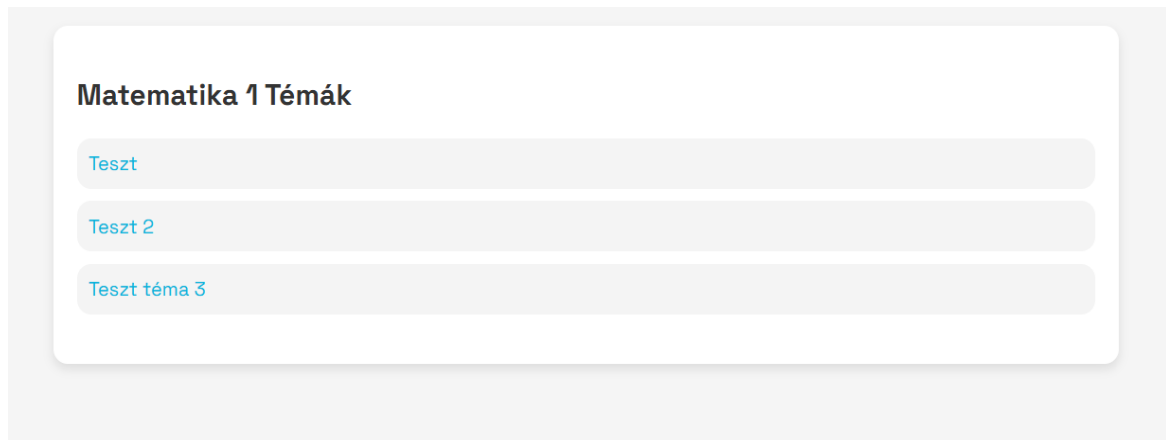
5. ábra Nincs találat

A projektben előre haladva arra a döntésre jutottunk, hogy megtartjuk a csempe nézetes megjelenítést a szakok, illetve a szakokon belül választható tantárgy tematikák között egyaránt (Kötelező tárgyak, Választható tantárgycsoport, Választható szakmai tantárgycsoport).

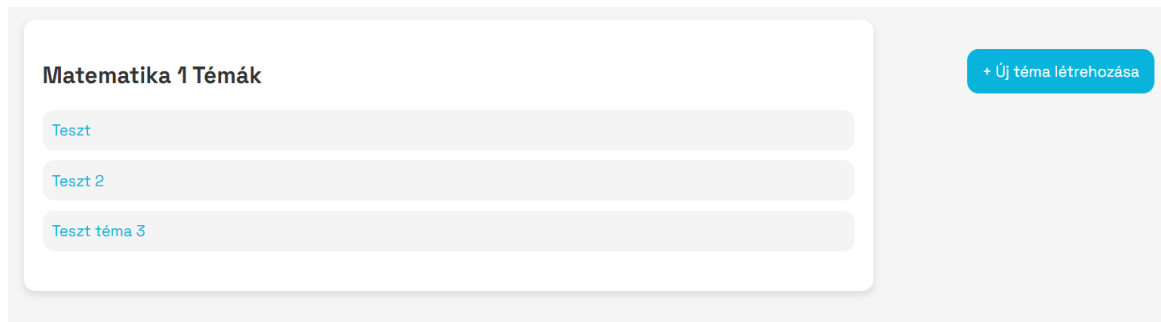


6. ábra Tantárgycsoportok

Amennyiben a Mérnökinformatikus szakot választjuk, azon belül pedig a „Kötelező tárgyak” kategóriát, megjelennek nekünk listászerűen egyes kötelező tárgyak. A Matematika 1.-et választva pedig megjelennek a tárgyhoz feltöltött témák ugyan úgy listába szedve.



8. ábra Tárgyhoz tartozó témák listája bejelentkezés előtt



7. ábra Tárgyhoz tartozó témák listája bejelentkezés után

A jobb oldalon található egy „+Új téma létrehozása” gomb, amellyel az adott tárgyhoz tudunk feltölteni új dolgokat. Egy új témának a hozzáadásakor kötelező kitölteni a „Téma neve” és a „Leírás” mezőket, mivel ez alapján tudják majd a jövőben a hallgatók beazonosítani az adott témát. Továbbá lehet csatolni hozzá fájlt is minden formátumban, nem alkalmaztunk külön megköttést ezekre. Miután minden kötelező mezőt kitöltöttünk, a „Küldés” gombra kattintva véglegesítjük a témánk létrehozásához szükséges adatokat. Az oldalnak van egy rendszergazdája/adminja, aki hozzáféréssel rendelkezik az adatbázishoz. Az oldalra mindenki szabadon tölthet fel bármit a bejelentkezés után, viszont a rendszergazda amennyiben hibát/kéretlen tartalmat vél felfedezni, szabadon törölheti a témát, és törölheti a felhasználót. Innentől kezdve, ha a kereső sávban valaki rákeres egy olyan szóra vagy kifejezésre, amelyet a témánk neve tartalmaz, akár meg is jelenhet a feljövő lehetőségek között.



Új téma létrehozása

Téma neve:
Vektorok

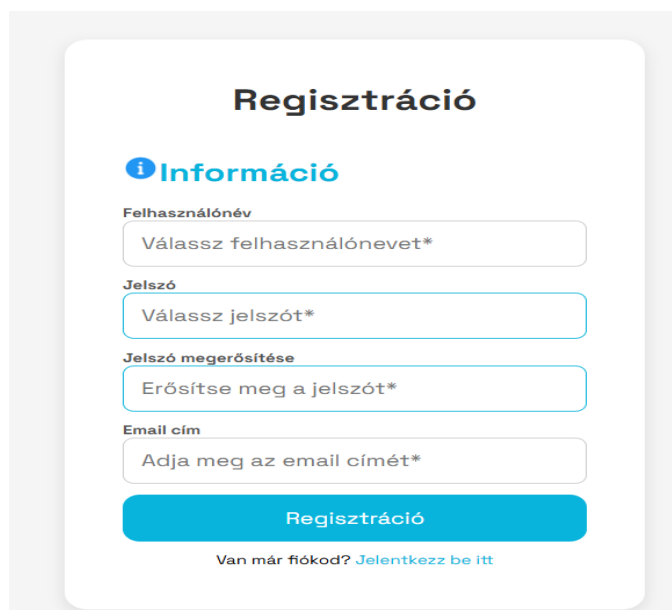
Leírás:
Az a téma a vektorokról szól.

Fájl kiválasztása vektorok.png

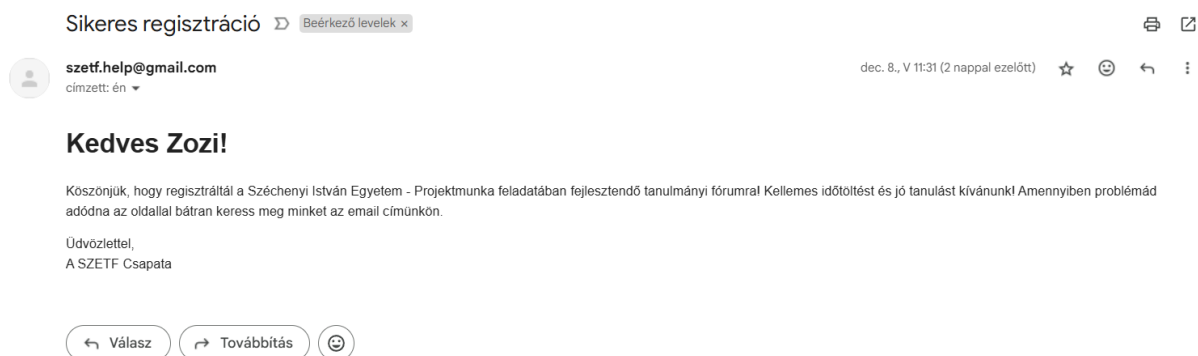
Küldés

9. ábra Téma létrehozása

Az oldalunkra opcionálisan regisztrálni lehet, amely annyit tesz lehetővé, hogy nem csak megnézni tudjuk a témákat, vagy böngészni az adott témák között, hanem fel is tölthetünk sajátot. Amennyiben ezzel a lehetőséggel élni szeretnénk, kattintsunk a „Bejelentkezés” felületen a „Regisztrálj itt” szövegre. Átirányít minket a regisztrációs felületre, ahol kötelezően meg kell adnunk egy egyedi felhasználónevet, egy jelszót (amelyhez segítséget a felhasználónév fölötti „Információ” lehetőségre kattintva találhatunk), illetve egy email címet, amelyre szeretnénk kapni a különböző üzeneteket az oldaltól. Amennyiben mindent helyesen kitöltöttünk, nyomjunk a regisztráció gombra, ezzel véglegesítve regisztrációunkat.

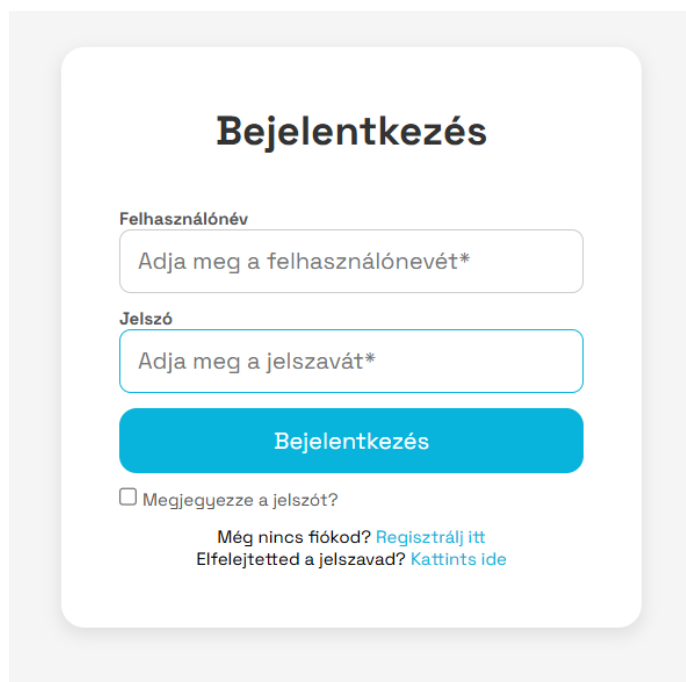


10. ábra Regisztrációs felület



11. ábra Regisztrációs email

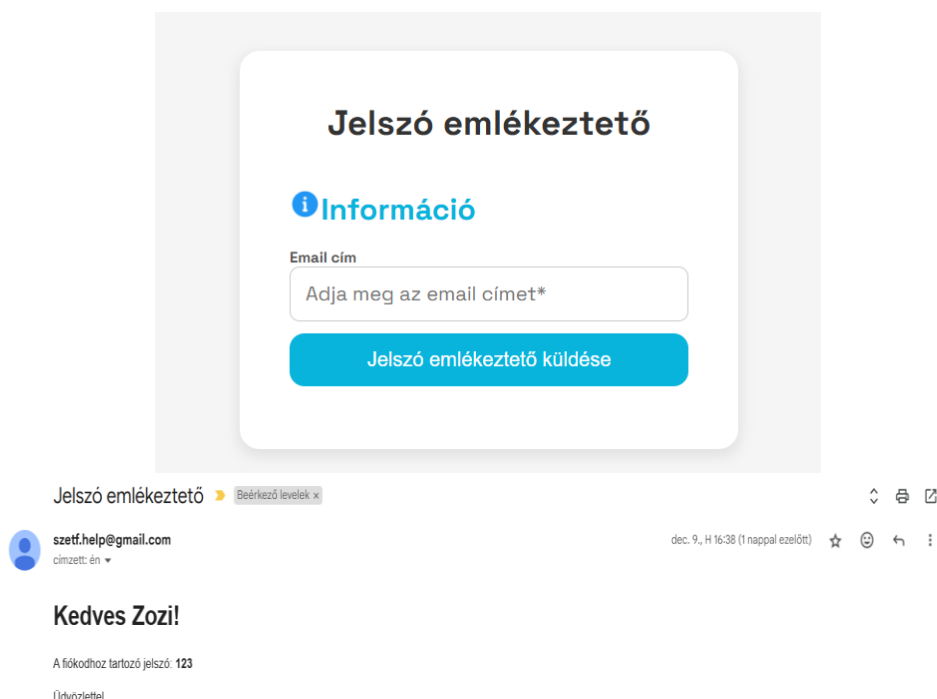
Ezt követően a „Bejelentkezés” oldalon be tudunk jelentkezni profilunkba.



The image shows a login form titled "Bejelentkezés". It contains two input fields: "Felhasználónév" (Username) and "Jelszó" (Password), both with placeholder text "Adja meg a felhasználónevét*" and "Adja meg a jelszavát*" respectively. Below the fields is a blue button labeled "Bejelentkezés". Under the button is a checkbox labeled "Megjegyezze a jelszót?". At the bottom, there are two links: "Még nincs fiókod? Regisztrálj itt" and "Elfelejtetted a jelszavad? Kattints ide".

12. ábra Bejelentkezés

Ha esetlegesen elfelejtettük a jelszavunkat, ebben az esetben a bejelentkezés gomb alatt megtaláljuk az „Elfelejtetted a jelszavad? Kattints ide” szöveget, kattintsunk a kékkel kijelölt szövegre, amely átirányít minket egy jelszó emlékeztető email üzenetet igénylő oldalra. Meg kell adnunk az oldalra regisztrált email címünket, majd a „Jelszó emlékeztető küldése” gombra kattintva érkezni fog egy email nekünk, amely tartalmazza a regisztrációkor általunk megadott jelszót.



The image shows a password reminder form and an email. The form is titled "Jelszó emlékeztető" and has an "Információ" icon. It contains an "Email cím" input field with placeholder text "Adja meg az email címet*" and a blue button labeled "Jelszó emlékeztető küldése". Below the form is an email interface. The email is from "szettf.help@gmail.com" and is addressed to "én". The subject is "Jelszó emlékeztető". The email body starts with "Kedves Zozil!" and mentions "A fiókodhoz tartozó jelszó: 123".

13. ábra Jelszó emlékeztető email

Back-End

A projekt feladat legelső fázisában olyan programnyelvben gondolkodtunk, amit már ismertünk és használtunk. Ezért esett a választásunk a C#-ra, viszont ez számunkra nem bizonyult jó megoldásnak az egyszerű kódolást tekintve illetve azt, hogy a kapcsolatok létrehozása mind a Front-end és Adatbázis között nehézkes volt. Ezért új irányt kerestünk, ami egyszerű, illetve rövid kódot tud eredményezni, így kisebb utánajárás és tájékozódást követően tétünk át JavaScriptre. Igaz mindannyiunk számára ismeretlen volt és teljes mértékben újdonságnak számított, de egyszerűsége és letisztultsága kecsegtető volt. Ezáltal új irányt vett a Back-end fejlesztése és ekkor már nagy lemaradás volt a Front-end előrehaladottságához képest. Ez annak is köszönhető volt, hogy nem ismertük a kód nyelvet, de online kurzusok, valamint alapvető programozási tudás segítségével könnyen megugrottuk ezt az akadályt.

A programozási nyelv megváltozása miatt a fejlesztői környezet is megváltozott Visual Studio helyett, váltottunk Visual Studio Code használatára.

Funkciók:

- Regisztráció, ezt a funkciót a Font-end fetchelés segítségével éri el és a User által megadott kötelező adatokat tárolja le(felhasználónév, email cím, jelszó, jelszó megerősítés). Első körben az adatbázisban lefuttat egy keresést, amely ellenőrzi az email címet, hogy létezik e már. Természetesen, ha már létezik, akkor nem engedi regisztrálni a felhasználót és erről tájékoztatja is a felhasználót, mivel hiba üzenetet ad. Ha nem létezik, akkor pedig letárolja az adatbázisba az adatokat és küld egy értesítő emailt a felhasználó számára a sikeres regisztrációról.

```
// Regisztráció végpont
app.post('/api/user/reg', async (req, res) => {
  const { username, email, password, confirm_password } = req.body;

  // Jelszavak ellenőrzése
  if (password !== confirm_password) {
    return res.status(400).json({ message: 'A jelszavak nem egyeznek.' });
  }

  try {
    // Ellenőrzik, hogy az email cím már regisztrálva van-e
    const [existingUser] = await db.promise().query('SELECT * FROM users WHERE email = ?', [email]);
    if (existingUser.length > 0) {
      return res.status(400).json({ message: 'Ez az e-mail cím már regisztrálva van.' });
    }

    // Új felhasználó hozzáadása az adatbázishoz
    await db.promise().query('INSERT INTO users (username, email, password, role) VALUES (?, ?, ?, ?)', [username, email, password, 'user']);

    // Email elküldése a felhasználónak
    const mailOptions = {
      from: 'szetf.help@gmail.com',
      to: email,
      subject: 'Sikeres regisztráció',
      html: `
        <h1>Kedves ${username}!</h1>
        <p>Köszönjük, hogy regisztráltál a Széchenyi István Egyetem tanulmányi fórumára!</p>
        <p>Bármilyen probléma esetén írj nekünk a szetf.help@gmail.com címre!</p>
        <p>Üdvözlettel,<br>SZETF Csapata</p>
      `
    };

    transporter.sendMail(mailOptions, (err, info) => {
      if (err) {
        console.error('Hiba az email küldése során:', err);
        return res.status(500).json({ message: 'A regisztráció sikerült, de az email küldése nem.' });
      }
      console.log('Email sikeresen elküldve:', info.response);
      res.status(201).json({ message: 'Sikeres regisztráció! Az email elküldve.' });
    });
  } catch (error) {
    console.error('Hiba a regisztráció során:', error);
    res.status(500).json({ message: 'Hiba történt a regisztráció során.' });
  }
});
```

- Regisztráció:**
 - Új API végpont definiálása POST metódussal a felhasználói regisztrációhoz.
`app.post('/api/user/reg', async (req, res) => {`
 - `const { username, email, password, confirm_password } = req.body;`
 - 1. A kliens által küldött adatokat kicsomagolja a kérés törzséből (`req.body`).
 - `if (password !== confirm_password) {`
 - 2. Ellenőrzi, hogy a megadott password és confirm_password megegyeznek-e.
 - `return res.status(400).json({ message: 'A jelszavak nem egyeznek.' });`
 - 3. Ha a jelszavak eltérnek, HTTP 400-as hibakódot és egy hibaüzenetet küld vissza.
 - `const [existingUser] = await db.promise().query('SELECT * FROM users WHERE email = ?', [email]);`
 - 4. Az adatbázisból lekérdezi, hogy a megadott email cím már regisztrált-e.
 - `if (existingUser.length > 0) {`

5. Ellenőrzi, hogy az email címhez tartozik-e már felhasználó.
 - return res.status(400).json({ message: 'Ez az e-mail cím már regisztrálva van.' });
 6. Ha az email cím már létezik, HTTP 400-as hibakódot és egy hibaüzenetet küld vissza.
 - await db.promise().query('INSERT INTO users (username, email, password, role) VALUES (?, ?, ?, ?)', [username, email, password, 'user']);
 7. Új felhasználót ad hozzá az adatbázishoz a megadott adatokkal.
 - const mailOptions = { ... }
 8. Beállítja az email tartalmát, címzettjét és tárgyát.
 - transporter.sendMail(mailOptions, (err, info) => {
 9. Elküldi az emailt a felhasználónak a konfigurált transporter segítségével.
 - if (err) {
 10. Ellenőrzi, hogy történt-e hiba az email küldése közben.
 - return res.status(500).json({ message: 'A regisztráció sikerült, de az email küldése nem.' });
 11. Ha hiba történt az email küldésekor, HTTP 500-as hibakódot és egy figyelmeztető üzenetet küld vissza.
 - console.log('Email sikeresen elküldve:', info.response);
 12. Ha az email sikeresen elküldésre került, kiírja az eredményt a konzolra.
 - res.status(201).json({ message: 'Sikeres regisztráció! Az email elküldve.' });
 13. Választ küld a kliensnek sikeres regisztrációról és email küldésről.
 - } catch (error) {
 14. Hibakezelés: kezeli az esetleges kivételeket.
 - console.error('Hiba a regisztráció során:', error);
 15. A hibát kiírja a konzolra.
 - res.status(500).json({ message: 'Hiba történt a regisztráció során.' });
 16. Általános hibaválaszt küld vissza a kliensnek HTTP 500-as státusszal.
- Bejelentkezés, ezt a funkciót a Front-end fetchelés segítségével éri el és a User által megadott kötelező adatokat keresi meg az adatbázisba (felhasználónév, jelszó). A bejelentkezés ablakban a felhasználó által megadott felhasználónévre, illetve jelszóra

```
// Bejelentkezés endpoint
app.post('/api/user/login', async (req, res) => {
  const { username, password } = req.body;

  try {
    const [users] = await db.promise().query('SELECT * FROM users WHERE username = ? AND password = ?', [username, password]);
    if (users.length === 0) {
      return res.status(401).json({ message: 'Hibás felhasználónév vagy jelszó.' });
    }
    res.status(200).json({ message: 'Sikeres bejelentkezés!', user: users[0] });
  } catch (error) {
    console.error('Hiba a bejelentkezés során:', error);
    res.status(500).json({ message: 'Hiba történt a bejelentkezés során.' });
  }
});

app.get('/api/user/status', (req, res) => {
  if (req.session.isLoggedIn) {
    res.status(200).json({
      isLoggedIn: true,
      userId: req.session.userId,
      role: req.session.role
    });
  } else {
    res.status(200).json({
      isLoggedIn: false
    });
  }
});

app.post('/api/user/logout', (req, res) => {
  req.session.destroy(err => {
    if (err) {
      console.error('Hiba a kijelentkezés során:', err);
      return res.status(500).json({ message: 'Hiba történt a kijelentkezés során.' });
    }
    res.status(200).json({ message: 'Sikeres kijelentkezés!' });
  });
});
```

keres rá és természetesen, ha nem létezik akkor nem engedi bejelentkezni, így nem is tudja elérni azokat a funkciókat (téma/fájl feltöltés, véleményezés).

- *Bejelentkezés:*

- `app.post('/api/user/login', async (req, res) => {`
- 17. Új API végpont definiálása POST módszerrel a bejelentkezéshez.
- `const { username, password } = req.body;`
- 18. A kliens által küldött username és password értékek kinyerése a kérés törzséből.
- `const [users] = await db.promise().query('SELECT * FROM users WHERE username = ? AND password = ?', [username, password]);`
- 19. Az adatbázisból lekérdezi, hogy létezik-e a felhasználó a megadott adatokkal.
- `if (users.length === 0) {`
- 20. Ellenőrzi, hogy a lekérdezés eredménye üres-e.
- `return res.status(401).json({ message: 'Hibás felhasználónév vagy jelszó.' });`
- 21. Ha nincs találat, HTTP 401-es hibakódot és hibaüzenetet küld vissza.
- `res.status(200).json({ message: 'Sikeres bejelentkezés!', user: users[0] });`
- 22. Ha sikeres a bejelentkezés, visszaküldi a felhasználó adatait HTTP 200-as státusszal.
- `} catch (error) {`
- 23. Hibakezelés kezdete.
- `console.error('Hiba a bejelentkezés során:', error);`
- 24. A konzolra kiírja a hibát, ha az adatbázis művelet közben hiba történt.
- `res.status(500).json({ message: 'Hiba történt a bejelentkezés során.' });`
- 25. Általános hibaválaszt küld vissza HTTP 500-as státusszal.

- *Státusz ellenőrzés:*

- `app.get('/api/user/status', (req, res) => {`
- 26. Új API végpont definiálása GET módszerrel a felhasználó státuszának lekérdezéséhez.
- `if (req.session.isLoggedIn) {`
- 27. Ellenőrzi, hogy a felhasználó be van-e jelentkezve a munkamenet alapján.
- `res.status(200).json({ isLoggedIn: true, userId: req.session.userId, role: req.session.role });`
- 28. Ha be van jelentkezve, visszaküldi a státuszt és a felhasználói adatokat.
- `} else {`
- 29. Ha nincs bejelentkezve, alternatív válasz küldése.
- `res.status(200).json({ isLoggedIn: false });`
- 30. Visszaküldi, hogy a felhasználó nincs bejelentkezve.

- *Kijelentkezés:*

- `app.post('/api/user/logout', (req, res) => {`
- 31. Új API végpont definiálása POST módszerrel a kijelentkezéshez.
- `req.session.destroy(err => {`
- 32. Munkamenet megszüntetése.
- `if (err) {`
- 33. Hibakezelés kezdete, ha a munkamenet lezárása sikertelen.
- `console.error('Hiba a kijelentkezés során:', err);`
- 34. A konzolra kiírja a munkamenet megszüntetése közben történt hibát.
- `return res.status(500).json({ message: 'Hiba történt a kijelentkezés során.' });`
- 35. Ha hiba történik, HTTP 500-as hibakóddal hibaüzenetet küld vissza.
- `res.status(200).json({ message: 'Sikeres kijelentkezés!' });`
- 36. Ha a kijelentkezés sikeres, HTTP 200-as státusszal visszaküldi a sikerről szóló üzenetet.
- 37.

- Jelszó emlékeztető küldése, ezt a funkciót a Font-end fetchelés segítségével éri el és a User által megadott email cím alapján dönti el, hogy létezik e már ehhez az email címhez profil, mindezt egy SQL lekérdezés alapján és ha sikeres a találat akkor küld egy email értesítőt amiben a User megkapja a jelszavát.

```
// Jelszó emlékeztető küldése
app.post('/api/reminder', async (req, res) => {
  const { email } = req.body;

  try {
    // Ellenőrizze, hogy létezik-e a felhasználó az adott e-maillal
    const [user] = await db.promise().query('SELECT username, password FROM users WHERE email = ?', [email]);

    if (user.length === 0) {
      return res.status(404).json({ message: 'A megadott e-mail cím nem található.' });
    }

    const { username, password } = user[0];

    // Email küldése a jelszóval
    const mailOptions = {
      from: 'szetf.help@gmail.com',
      to: email,
      subject: 'Jelszó emlékeztető',
      html: `
        <h1>Kedves ${username}!</h1>
        <p>A fiókjához tartozó jelszó: <strong>${password}</strong></p>
        <p>Üdvözlettel,<br>SZETF Csapata</p>
      `
    };

    transporter.sendMail(mailOptions, (err, info) => {
      if (err) {
        console.error('Hiba az email küldése során:', err);
        return res.status(500).json({ message: 'Hiba történt az email küldése során.' });
      }
      console.log('Email sikeresen elküldve:', info.response);
      res.status(200).json({ message: 'A jelszó emlékeztető email sikeresen elküldve.' });
    });
  } catch (error) {
    console.error('Hiba a jelszó emlékeztető során:', error);
    res.status(500).json({ message: 'Hiba történt a jelszó emlékeztető során.' });
  }
});
```

- *Jelszó emlékeztető küldése:*

- `app.post('/api/reminder', async (req, res) => {`
- 38. Új API végpont definiálása POST metódussal a jelszó emlékeztetőhöz.
- `const { email } = req.body;`
- 39. A kérés törzséből (`req.body`) kinyeri a megadott email címet.
- `const [user] = await db.promise().query('SELECT username, password FROM users WHERE email = ?', [email]);`
- 40. Az adatbázisból lekérdezi a felhasználó adatait a megadott email cím alapján (`username, password`).
- `if (user.length === 0) {`
- 41. Ellenőrzi, hogy az email cím létezik-e az adatbázisban.
- `return res.status(404).json({ message: 'A megadott e-mail cím nem található.' });`
- 42. Ha az email cím nem létezik, HTTP 404-es hibakódot és hibaüzenetet küld vissza.
- `const { username, password } = user[0];`
- 43. Kinyeri a lekérdezés eredményéből a felhasználónevet és a jelszót.
- `const mailOptions = { ... };`
- 44. Létrehozza az email konfigurációját, beleértve a címzettet (`email`), a tárgyat és az üzenet tartalmát.
- `transporter.sendMail(mailOptions, (err, info) => {`
- 45. Elküldi az emailt a felhasználónak a konfigurált transporter segítségével.
- `if (err) {`
- 46. Ellenőrzi, hogy történt-e hiba az email küldése során.
- `console.error('Hiba az email küldése során:', err);`

- 47. Ha hiba történt, a konzolra kiírja az email küldési hibát.
 - return res.status(500).json({ message: 'Hiba történt az email küldése során.' });
 - 48. Ha hiba történik az email küldésekor, HTTP 500-as hibakóddal és hibaüzenettel válaszol.
 - console.log('Email sikeresen elküldve:', info.response);
 - 49. Ha az email sikeresen elküldésre került, kiírja az eredményt a konzolra.
 - res.status(200).json({ message: 'A jelszó emlékeztető email sikeresen elküldve.' });
 - 50. Sikeres email küldés esetén HTTP 200-as státusszal visszaküldi a sikerről szóló üzenetet.
 - } catch (error) {
 - 51. Hibakezelés kezdete: bármilyen más hiba esetére.
 - console.error('Hiba a jelszó emlékeztető során:', error);
 - 52. Ha a folyamat során hiba történik (pl. adatbázis művelet), a konzolra kiírja a hibát.
 - res.status(500).json({ message: 'Hiba történt a jelszó emlékeztető során.' });
 - 53. Általános hibaválaszt küld vissza HTTP 500-as státusszal.
- Téma létrehozása, ezt a funkciót a Font-end fetchelés segítségével éri el és a User által megadott adatokat tárolja le(téma neve, fájl(Word,PPT,PDF,stb.), leírás). A letárolásban a fájlnev alapján tárolja le a feltöltött fájlokat, neveket a megadott tantárgyhoz.

```
// Új téma létrehozása
app.post('/api/targyak', upload.single('file'), async (req, res) => {
  console.log('Feltöltött fájl:', req.file); // Ellenőrizd a konzolon
  console.log('Kapott adatok:', req.body); // Ellenőrizd a szöveges adatokat

  const { temaNev, leiras } = req.body;
  const filePath = req.file ? `uploads/${req.file.filename}` : null;

  if (!temaNev || !leiras) {
    return res.status(400).json({ message: 'Hiányzó adatok.' });
  }

  try {
    await db.promise().query('INSERT INTO targyak (title, description, file) VALUES (?, ?, ?)', [temaNev, leiras, filePath]);
    res.status(201).json({ message: 'Téma sikeresen létrehozva!' });
  } catch (error) {
    console.error('Hiba a téma létrehozása során:', error);
    res.status(500).json({ message: 'Hiba történt a téma létrehozása során.' });
  }
});

app.get('/api/topics', async (req, res) => {
  try {
    const [rows] = await db.promise().query('SELECT id, title AS tema_name FROM targyak WHERE status = "approved"');
    res.status(200).json(rows);
  } catch (error) {
    console.error('Hiba a témák lekérzése során:', error);
    res.status(500).json({ message: 'Hiba történt a témák lekérzése során.' });
  }
});

app.post('/api/topics', async (req, res) => {
  const { title, description } = req.body;
  try {
    await db.promise().query('INSERT INTO targyak (title, description, status) VALUES (?, ?, "pending")', [title, description]);
    res.status(201).json({ message: 'Téma létrehozva, adminisztrátori jóváhagyás szükséges!' });
  } catch (error) {
    console.error('Hiba a téma létrehozása során:', error);
    res.status(500).json({ message: 'Hiba történt a téma létrehozása során.' });
  }
});
```

- *Új téma létrehozása:*
 - app.post('/api/targyak', upload.single('file'), async (req, res) => {
 - 54. Új végpont definiálása POST módszerrel, amely fájlfeltöltést és téma létrehozást kezel.

- console.log('Feltöltött fájl:', req.file);
- 55. Konzolra írja a feltöltött fájl adatait (hibakeresés céljából).
- console.log('Kapott adatok:', req.body);
- 56. Konzolra írja a kliens által küldött szöveges adatokat.
- const { temaNev, leiras } = req.body;
- 57. Kinyeri a téma nevét (temaNev) és leírását (leiras) a kérés törzséből.
- const filePath = req.file ? 'uploads/' + req.file.filename : null;
- 58. Ha van feltöltött fájl, elmenti annak elérési útját; egyébként null értéket állít be.
- if (!temaNev || !leiras) {
- 59. Ellenőrzi, hogy a kötelező adatok meg lettek-e adva.
- return res.status(400).json({ message: 'Hiányzó adatok.' });
- 60. Ha hiányoznak az adatok, HTTP 400-as hibakóddal hibaüzenetet küld vissza.
- await db.promise().query('INSERT INTO targyak (title, description, file) VALUES (?, ?)', [temaNev, leiras, filePath]);
- 61. Az adatbázisba menti az új téma adatait (cím, leírás és fájl elérési út).
- res.status(201).json({ message: 'Téma sikeresen létrehozva!' });
- 62. Sikeres mentés esetén HTTP 201-es státusszal visszaküldi a sikerről szóló üzenetet.
- } catch (error) {
- 63. Hibakezelés kezdete.
- console.error('Hiba a téma létrehozása során:', error);
- 64. Ha hiba történik, a konzolra írja a hiba részleteit.
- res.status(500).json({ message: 'Hiba történt a téma létrehozása során.' });
- 65. Általános hibaválaszt küld vissza HTTP 500-as státusszal.
- 66.
- Keresési sáv, , ezt a funkciót a Font-end fetchelés segítségével éri el és a User által a keresőbe beírt tantárgyra/kurzusra keres rá és ennek segítségével adja vissza a Back-end az Adatbázisból a keresett tantárgyat/kurzust, illetve a hozzá tartozó témákat.

```
// Keresési végpont témákhoz
app.get('/api/topics/search', async (req, res) => {
  const query = req.query.query;

  if (!query) {
    return res.status(400).json({ message: 'Keresési kifejezés hiányzik.' });
  }

  try {
    // SQL lekérdezés a címek keresésére
    const [results] = await db.promise().query(
      'SELECT id, title FROM targyak WHERE title LIKE ?',
      [`%${query}%`]
    );

    res.status(200).json(results);
  } catch (error) {
    console.error('Hiba a keresési lekérdezés során:', error);
    res.status(500).json({ message: 'Hiba történt a keresés során.' });
  }
});
```

- *Keresés témákhoz:*
 - app.get('/api/topics/search', async (req, res) => {
 - 67. Új API végpont definiálása GET módszerrel, amely a témák keresését kezeli.

- `const query = req.query.query;`
- 68. Kinyeri a keresési kifejezést az URL-ben található lekérdezési paraméterek közül (`req.query`).
- `if (!query) {`
- 69. Ellenőrzi, hogy a keresési kifejezés (`query`) meg lett-e adva.
- `return res.status(400).json({ message: 'Keresési kifejezés hiányzik.' });`
- 70. Ha nincs megadva keresési kifejezés, HTTP 400-as hibakóddal hibaüzenetet küld vissza.
- `const [results] = await db.promise().query('SELECT id, title FROM targyak WHERE title LIKE ?, [%${query}]');`
- 71. SQL lekérdezést hajt végre az adatbázisban:
- 72. A `title` mezőben megkeresi azokat a rekordokat, amelyek tartalmazzák a keresési kifejezést (LIKE operátor segítségével).
- 73. A keresési kifejezést `%` karakterekkel veszi körül, hogy rugalmas találatokat biztosítson.
- `res.status(200).json(results);`
- 74. A lekérdezés eredményét HTTP 200-as státusszal JSON formátumban visszaküldi.
- `} catch (error) {`
- 75. Hibakezelés kezdete, ha a keresési művelet során hiba történik.
- `console.error('Hiba a keresési lekérdezés során:', error);`
- 76. Konzolra írja a hibát, ha a lekérdezés végrehajtása közben hiba történt.
- `res.status(500).json({ message: 'Hiba történt a keresés során.' });`
- 77. Általános hibaválaszt küld vissza HTTP 500-as státusszal.

A Back-end működését tekintve csak, funkciókat működtet, amelyeket a Front-end egyes menü pontjai, kereső sávjai és funkciói működtetik egy hívás segítségével, valamint, ezen funkciók által nyer összeköttetést az Adatbázissal is.

Könyvtárak importálása

- **express:** A Node.js egyik legnépszerűbb webes keretrendszere, amely segít HTTP szerverek létrehozásában.
- **mysql2:** Egy gyors és könnyű MySQL kliens, amely lehetővé teszi az adatbázishoz való kapcsolódást és annak kezelését.
- **body-parser:** A HTTP kérésekben található adatokat (pl. POST kérésben a JSON-t) elemzi.
- **multer:** A fájlok feltöltését segítő middleware, amelyet gyakran használnak képek vagy más fájlok kezelésére.
- **path:** Node.js beépített modulja, amely az útvonalak kezelésére szolgál.
- **nodemailer:** Email küldésére használt modul.
- **express-session:** A munkamenet-kezelésért felelős modul, amely lehetővé teszi, hogy adatokat tároljunk a felhasználói munkamenet során.

Modulok importálása

A kód az alkalmazáshoz szükséges csomagokat importálja. Ezek a csomagok teszik lehetővé a webalkalmazás működését.

Express alkalmazás inicializálása

Az app változó tartalmazza az Express alkalmazást, amelyet a későbbiekben konfigurálunk és használunk HTTP kérések kezelésére.

Body-parser middleware konfigurálása

- `bodyParser.json()`: Lehetővé teszi a JSON adatok elemzését a beérkező HTTP kérések törzsében.
- `bodyParser.urlencoded({ extended: true })`: Elemzi az „application/x-www-form-urlencoded” formátumú adatokat. Az `extended: true` opció lehetővé teszi a komplexebb objektumok kezelését.

Statikus fájlok kiszolgálása

- A `path.join` segítségével az aktuális fájl elérési útja alapján egy abszolút útvonalat készítünk.
- Az `express.static` middleware a megadott könyvtárban található statikus fájlokat (pl. HTML, CSS, JavaScript, képek) szolgálja ki. Ebben az esetben a `public` mappa tartalmát.

Session kezelés konfigurálása

- `secret`: Egy titkos kulcs, amelyet a session aláírására használnak. Ennek egyedi és biztonságos értéket kell kapnia.
- `resave`: Ha `false`, akkor a munkamenet csak akkor mentődik újra, ha változás történt benne.
- `saveUninitialized`: Ha `true`, akkor az üres session-öket is menti.
- `cookie`:
 - `secure`: Ha `true`, akkor a munkamenet-cookie-k csak HTTPS-en keresztül érhetők el.
 - `httpOnly`: Ha `true`, akkor a cookie-k csak HTTP-n keresztül érhetők el, és nem érhetők el JavaScript-ből.

Ez a middleware kezeli a felhasználói munkameneteket, például amikor a felhasználó belép egy weboldalra, és a szervernek emlékeznie kell rá.

Nodemailer konfiguráció

A Nodemailer egy olyan Node.js modul, amely lehetővé teszi az e-mailek küldését SMTP protokollon keresztül. A jelenlegi kód konfigurációja a Gmail szolgáltatást használja az e-mailek küldéséhez.

Multer konfiguráció

A Multer egy middleware, amelyet a fájlok HTTP kéréseken keresztüli kezelésére használnak (főként fájlfeltöltésre). A jelenlegi kód a feltöltött fájlokat a szerveren belül egy `uploads/` mappába menti.

Adatbázis kapcsolat beállítása

A MySQL adatbázis konfigurációja és a szerverhez való csatlakozás.

Regisztrációs végpont

Egy API végpont, amely a felhasználók regisztrációját kezeli.

Jelszavak egyezősége

Ellenőrzi, hogy a megadott jelszó és annak megerősítése azonosak-e.

E-mail cím megléte

Ellenőrzi, hogy a megadott e-mail cím már szerepel-e az adatbázisban.

Új felhasználó adatainak mentése az adatbázisba

- A felhasználónév, e-mail cím, jelszó, és alapértelmezett szerepkör kerül az adatbázis users táblájába.
- Az adatbázis-művelet aszinkron módon történik.

E-mail értesítés küldése

- Az új felhasználó számára egy e-mail értesítést küld a rendszer a sikeres regisztrációról.
- A folyamat a Nodemailer könyvtáron keresztül valósul meg.
- Hibakezelés: Az adatbázisba írás és az e-mail küldés során felmerülő hibák külön-külön kerülnek kezelésre.

Felhasználó ellenőrzése az adatbázisban

- Az e-mail cím alapján lekérdezi a felhasználónevet és a jelszót.
- Ha az e-mail cím nem található, hibaüzenetet ad vissza.

E-mail küldése a jelszóval

- Az adatbázisból lekért adatok (felhasználónév és jelszó) alapján generált e-mailt küld a felhasználónak.
- A folyamat a Nodemailer könyvtár segítségével valósul meg.
- Hibakezelés: Az adatbázis-műveletek és az e-mail küldés során fellépő hibák külön-külön kerülnek kezelésre.

Felhasználói adatok fogadása

- Az API a POST kéréssel küldött adatokat (username, password) fogadja.
- A Body Parser middleware biztosítja az adatok elérhetőségét a kérés testében.

Adatbázis-lekérdezés

- A megadott felhasználónév és jelszó alapján ellenőrzi, hogy a felhasználó létezik-e az adatbázisban.

Eredmény ellenőrzése

- Ha nincs egyező rekord: 401-es státuszkód visszaküldése, amely hitelesítési hibára utal.
- Ha a felhasználó megtalálható: 200-as státuszkód visszaküldése, és a felhasználó adatainak megjelenítése.
- Hibakezelés: Ha bármilyen hiba történik a folyamat során, a rendszer 500-as státuszkóddal válaszol, amely a belső szerverhiba jelzése.

Új téma létrehozása

- Végpont: /api/targyak
- HTTP Módszer: POST
- Funkció: Ez a végpont lehetővé teszi, hogy új témát hozzunk létre, amelyhez fájl is csatolható.

Kérelmi adat (Body):

- **temaNev:** A téma neve, amelyet a felhasználó ad meg (pl. "Matematika").
- **leiras:** A téma rövid leírása (pl. "A matematika alapjai").
- **file** (opcionális): Egy fájl, amelyet a felhasználó feltölt a téma mellé (pl. PDF dokumentum vagy kép).

Keresés a témák között

- **Végpont:** /api/topics/search
- **HTTP Módszer:** GET
- **Funkció:** Ez a végpont lehetővé teszi a felhasználók számára, hogy keresést végezzenek a témák között, a cím alapján.

Kérelmi adat (Query paraméterek):

- **query:** A keresési kifejezés, amelyet a felhasználó ad meg (pl. "matematika").

Témák listázása

- **Végpont:** /api/topics
- **HTTP Módszer:** GET
- **Funkció:** Ez a végpont lehetővé teszi az összes téma listázását, a témák ID-jával és nevével.

Kérelmi adat:

- Nincs szükség kérés törzsre (query parameters vagy request body nincs).

Téma részleteinek lekérése

- **Végpont:** /api/topic/:id
- **HTTP Módszer:** GET
- **Funkció:** Ez a végpont lehetővé teszi egy adott téma részleteinek lekérését a téma id alapján.

Kérelmi adat (URL paraméter):

- **:id:** A lekérdezni kívánt téma egyedi azonosítója.

Szerver indítása

- A szerver a 5001 porton indul el, és figyeli a bejövő kéréseket.

Fejlesztési eszközök

- Verziókezelés: GitHub, Git
- Operációs rendszerek: Windows 11, Windows 10 Pro
- Fejlesztési környezetek: VSCode, VS, MySQL, XAMPP, Node.js
- Diagramok készítéséhez használt eszközök: dbdigarm.io,
- Dokumentáláshoz használt eszközök: MS Word, Discord, GitHub
- Teszteléshez használt eszközök: XAMPP, Node.js, MySQL
- Debugger eszközök: VSCode
- Kommunikációs platformok: Discord, Messenger, MS Teams, Google Meet